

System for Cross-domain Identity Management (SCIM 2.0) is the standard protocol for synchronizing user identity data from Entra ID (or ADFS) to application attribute stores. Rather than embedding all attributes in JWT tokens (which hits size limits), SCIM pushes attributes to a local DynamoDB store that the PIP queries at authorization time.

SCIM · Nested Groups · Role Explosion · Cedar Templates · OPA GitOps · WASM · Partial Evaluation

1.1 SCIM Architecture for Enterprise AI

Microsoft Entra ID ■ ■ SCIM 2.0 Provisioning (push on change) ■ Endpoint: <https://api.bank.com/scim/v2> ▼ SCIM Receiver Lambda ■ Validates SCIM bearer token ■ Transforms SCIM schema → Enterprise schema ▼ DynamoDB User Attribute Store ■ Partition: `userId` ■ Contains: `department`, `costCenter`, `clearanceLevel`, ■ `geography`, `legalEntity`, `manager`, ■ `projectMemberships`, `tradingDeskId`, ■ `regulatoryCaptures`, `dataAccessScopes` ■ ■ TTL: Not set (SCIM manages lifecycle) ■ GSI: `by-department`, `by-geography` ▼ PIP Lookup Lambda (at authorization time) ■ Cache: `ElasticCache Redis 300s TTL` ■ Enriches canonical claims with SCIM attributes ▼ Cedar Authorization Context (enriched)

1.2 SCIM Receiver Implementation

```
# SCIM 2.0 Receiver – Lambda function
import json
import boto3
import hashlib
import dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('user-attributes')

def handler(event, context):
    path = event['path']
    method = event['httpMethod']
    body = json.loads(event.get('body', '{}'))

    # SCIM 2.0 endpoint routing
    if '/Users' in path and method == 'POST':
        return provision_user(body)
    elif '/Users/' in path and method == 'PUT':
        return update_user(path, body)
    elif '/Users/' in path and method == 'DELETE':
        return deprovision_user(path)
    elif '/Users' in path and method == 'GET':
        return list_users(event.get('queryStringParameters', {}))

    def provision_user(scim_user: dict) -> dict:
        # Transform SCIM schema to enterprise schema
        enterprise_user = {
            'userId': scim_user['id'], # Entra ID object ID 'upn':
            scim_user['userName'], 'displayName': scim_user['displayName'], # SCIM Enterprise Extension
            'department': scim_user.get('urn:ietf:params:scim:schemas:extension:enterprise:2.0:User', {}
            ).get('department', 'UNKNOWN'), 'costCenter': scim_user.get(
            'urn:ietf:params:scim:schemas:extension:enterprise:2.0:User', {} ).get('costCenter', ''),
            'geography': extract_geography(scim_user), 'clearanceLevel': extract_clearance(scim_user),
            'legalEntity': extract_legal_entity(scim_user), # Custom bank extension
            'tradingDeskId': scim_user.get('urn:bank:scim:extension:trading', {} ).get('deskId', ''), 'projectMemberships':
            [], # Populated by group sync
            'lastSyncedAt': get_iso_timestamp(), 'active': scim_user.get('active', True) }

        # Write to DynamoDB table
        table.put_item(Item=enterprise_user)
        return { 'statusCode': 201, 'body': json.dumps({'id': enterprise_user['userId']}) }

    def update_user(path: str, scim_user: dict) -> dict:
        user_id = path.split('/')[-1] # PATCH or PUT updates – only update changed attributes
        updates = build_update_expression(scim_user)
        table.update_item(
            Key={'userId': user_id},
            UpdateExpression=updates['expression'],
            ExpressionAttributeValues=updates['values'],
            ConditionExpression='attribute_exists(userId)' # Safety check
        )
        # Invalidate ElasticCache entry for this user
        invalidate_pip_cache(user_id)
        return { 'statusCode': 200, 'body': json.dumps({'id': user_id}) }

    def deprovision_user(path: str) -> dict:
        user_id = path.split('/')[-1] # Mark as inactive (don't delete – audit requirement)
        table.update_item(
            Key={'userId': user_id},
            UpdateExpression='SET active = :false, deactivatedAt = :ts',
            ExpressionAttributeValues={':false': False, ':ts': get_iso_timestamp()}
        )
        invalidate_pip_cache(user_id)
        return { 'statusCode': 204 }
```

2. Nested Group Resolution Algorithm

Microsoft Entra ID supports deeply nested security groups. A user's effective role set is the transitive closure of all direct and indirect group memberships. This computation must be done efficiently at authorization time.

2.1 The Transitive Closure Problem

ENTRA ID GROUP HIERARCHY (example): GlobalAdmins ■■■ EMEA_Admins ■■■ UK_Admins ■■■ London_Payments_Admins ■■■ john.smith ← User JWT contains ONLY direct memberships: { "groups": ["London_Payments_Admins_GUID"] } BUT effective capabilities should include: • can_approve_payment (from London_Payments_Admins) • can_view_emea_data (from EMEA_Admins) • can_access_global_reports (from GlobalAdmins) WITHOUT nested resolution, John is missing inherited capabilities. WITH resolution, we flatten the hierarchy into the capability set.

2.2 Efficient Resolution with Caching

```
# Nested group resolver with DFS and caching
import redis
import json
import boto3
from typing import Set

r = redis.Redis(host='elasticsearch-endpoint', port=6379, db=0)
graph_client = None # Microsoft Graph API client

def resolve_transitive_groups(user_id: str, direct_groups: list) -> Set[str]:
    cache_key = f"groups:{user_id}" # Check cache first (TTL = 300 seconds)
    cached = r.get(cache_key)
    if cached:
        return set(json.loads(cached)) # BFS/DFS traversal of group hierarchy
    visited = set()
    queue = list(direct_groups)
    while queue:
        group_guid = queue.pop(0)
        if group_guid in visited:
            continue
        visited.add(group_guid)
        # Get parent groups (transitive membership)
        parent_groups = get_parent_groups(group_guid)
        for parent in parent_groups:
            if parent not in visited:
                queue.append(parent)
    # Cache the resolved set
    r.setex(cache_key, 300, json.dumps(list(visited)))
    return visited

def resolve_capabilities(group_guids: Set[str]) -> list:
    # Map group GUIDs to business capabilities.
    cache_key = f"caps:{hash_set(group_guids)}"
    cached = r.get(cache_key)
    if cached:
        return json.loads(cached)
    capabilities = set()
    for guid in group_guids:
        group_name = resolve_group_name(guid) # GUID → human name
        caps = GROUP_TO_CAPABILITY_MAP.get(group_name, [])
        capabilities.update(caps)
    result = list(capabilities)
    r.setex(cache_key, 300, json.dumps(result))
    return result

# The mapping table: loaded from DynamoDB (managed by IAM team)
GROUP_TO_CAPABILITY_MAP = {
    "London_Payments_Admins": ["can_approve_payment", "can_view_payments"],
    "EMEA_Payments_Admins": ["can_approve_payment", "can_view_emea_data"],
    "GlobalAdmins": ["can_access_global_reports", "can_view_all_data"],
    "Finance_Readonly": ["can_view_financial_data"],
    "DBA_Production": ["can_query_production_db"],
    "Compliance_Auditors": ["can_export_audit_log", "can_view_all_data"],
    # ... hundreds more mappings managed in DynamoDB
}
```

2.3 Role Explosion Mitigation

Large enterprises often have thousands of AD groups. Without mitigation, the JWT groups claim and the capability set both explode in size, causing performance and token size problems.

Problem	Scale	Mitigation Strategy	Implementation
Too many groups in JWT	>200 groups → JWT > 4KB, HTTP header overflow	Reference token + PIP lookup	JWT contains jti only; full groups loaded from DynamoDB on demand
Group GUID explosion	Thousands of GUIDs with no semantic meaning	GUID → name cache in ElastiCache	Pre-loaded at normalization service startup; refresh every 5 min
Capability set explosion	1000 groups → 3000 capabilities	Scope scoping: only capabilities relevant to the requested resource	Lazy capability resolution: only resolve groups relevant to the action being authorized

Problem	Scale	Mitigation Strategy	Implementation
Transitive resolution latency	10+ levels deep → 50+ MS Graph API calls	Pre-computed transitive closure in DynamoDB (SCIM-updated)	SCIM sync writes full transitive group list; no real-time traversal needed
Policy evaluation with huge capability sets	Cedar policy with contains on 3000-element set	Segment capabilities by domain: finance_caps, hr_caps, etc.	Cedar context uses typed sets per domain, not one flat list

3. Cedar Policy Templates & Delegated Administration

Cedar Policy Templates allow parameterized policies to be instantiated for many entities without writing individual policies for each. This is critical for multi-tenant SaaS deployments and delegated administration models.

3.1 Cedar Policy Templates

```
// Cedar Policy Template – parameterized by ?principal and ?resource // Template:
resource-owner-access // Allows a specific user to access a specific resource they own
@id("resource-owner-template") permit( principal == ?principal, action in
[BankAI::Action::"ReadDocument", BankAI::Action::"EditDocument"], resource == ?resource ); //
Template instantiation (via AWS SDK or AVP Console): // For John's M&A; document: // ?principal
= BankAI::User::"john.smith" // ?resource = BankAI::Document::"doc-ma-2025-001" // This
generates a SPECIFIC policy without rewriting the template. // 10,000 users × 10 documents =
100,000 policies // But only 1 template to maintain! // Another template: project member access
@id("project-member-template") permit( principal == ?principal, action in
[BankAI::Action::"ReadDocument", BankAI::Action::"CommentDocument"], resource in
BankAI::Project::"?project" // All docs in project ) when { principal.active == true }; //
Instantiated for each project member: // ?principal = BankAI::User::"jane.doe" // ?project =
BankAI::Project::"project-phoenix" // Template-based policies can be managed programmatically:
avp_client.create_policy( policyStoreId=POLICY_STORE_ID, definition={ 'templateLinked': {
'policyTemplateId': 'resource-owner-template', 'principal': { 'entityType': 'BankAI::User',
'entityId': 'john.smith' }, 'resource': { 'entityType': 'BankAI::Document', 'entityId':
'doc-ma-2025-001' } } } )
```

3.2 Delegated Administration with Cedar

Delegated administration allows department heads or project leads to manage authorization within their scope without touching the central policy store. Cedar's entity hierarchy and template system enable this safely:

```
// Delegated Admin Model: // A Finance Manager can grant/revoke access to Finance documents //
WITHOUT being able to affect HR or Legal documents // 1. Finance Manager is an Admin within
Finance scope only permit( principal == BankAI::User::"sarah.jones", action ==
BankAI::Action::"ManageAccess", resource in BankAI::DocumentScope::"Finance" ) when {
principal.capabilities.contains("can_admin_finance_scope") }; // 2. The delegated admin
workflow: // a. Sarah (Finance Manager) requests to grant John access to doc-X // b. API
validates Sarah has ManageAccess to Finance scope // c. System creates template-linked policy
for John + doc-X // d. Cedar evaluates: John can now access doc-X // e. CloudTrail records:
sarah.jones granted john.smith access to doc-X // 3. Guardrails: Delegated admins CANNOT: // •
Grant access beyond their scope (Finance) // • Grant access to classified > their own clearance
// • Grant ADMIN permissions (only read/edit) // • Grant access without MFA challenge forbid(
principal, action == BankAI::Action::"ManageAccess", resource ) when { // Cannot elevate beyond
own clearance resource.classification > principal.clearanceLevel } unless {
principal.capabilities.contains("can_admin_global_scope") };
```

3.3 Cedar Policy Versioning Strategy

Version Type	Approach	Rollback Mechanism	Use Case
Major version (breaking)	New policy store; traffic migration via canary	Switch canary back to 0%	Fundamental model change (e.g. new entity type)

Version Type	Approach	Rollback Mechanism	Use Case
Minor version (additive)	Add new permit policies; shadow evaluate first	Delete new policies	New capability or resource type
Patch (fix)	Replace existing policy; validate with test suite	Git revert + pipeline redeploy	Bug fix in existing policy logic
Emergency (hotfix)	Direct AVP update with dual approver (break-glass)	Immediate AVP delete or disable	Active security incident requiring immediate policy change

4. OPA Advanced Patterns

4.1 OPA Bundle Distribution via S3 + GitOps

```
# OPA Bundle Server on S3 – GitOps workflow # Directory structure: # /policies/ # bundle.tar.gz
(built by CI/CD from Git) # /policies/ # kubernetes/ # admission.rego # network.rego #
terraform/ # aws_resources.rego # security.rego # agent/ # tool_authorization.rego #
memory_access.rego # /data/ # tool_capability_map.json # risk_thresholds.json #
approved_images.json # /.manifest # { "revision": "git-sha-abc123", "roots": ["policies",
"data"] } # OPA configuration (opa-config.yaml): services: s3-bundle-server: url:
https://s3.amazonaws.com/enterprise-opa-bundles credentials: s3_signing:
environment_credentials: {} bundles: enterprise: service: s3-bundle-server resource:
/enterprise/bundle.tar.gz polling: min_delay_seconds: 30 max_delay_seconds: 60 signing: keyid:
"opa-bundle-signing-key" # Verify bundle signature decision_logs: service: decision-log-service
reporting: min_delay_seconds: 5 max_delay_seconds: 10 buffer_size_limit_bytes: 65536 status:
service: s3-bundle-server # Status API for health monitoring # Bundle signing (in CI/CD
pipeline): # opa build -b policies/ data/ --signing-key bundle-signing.pem # aws s3 cp
bundle.tar.gz s3://enterprise-opa-bundles/enterprise/bundle.tar.gz
```

4.2 OPA Partial Evaluation for Database-Level Authorization

OPA's partial evaluation feature compiles a Rego policy against known inputs to produce a residual policy — typically a set of conditions that can be directly applied as a SQL WHERE clause or OpenSearch filter. This is the most powerful OPA pattern for data-level authorization:

```
# Rego policy for document access package data.documents import future.keywords.if # Full
policy allow if { input.user.capabilities[_] == "can_view_financial_data"
document_permitted(input.document) } document_permitted(doc) if { doc.classification <=
input.user.clearance_level doc.tenant_id == input.user.tenant_id not doc.embargo_active } #
Partial evaluation query: # Which document conditions apply for user john.smith? # OPA produces
RESIDUAL POLICY: # { # "result": [ # { "expressions": [ # { "value": true }, # { "conditions": [
# "doc.classification <= 3", // L3 clearance # "doc.tenant_id == 'bank-prod'", #
"doc.embargo_active == false" # ]} # ]} # ] # } # This RESIDUAL is compiled to SQL: # WHERE
classification <= 3 # AND tenant_id = 'bank-prod' # AND embargo_active = false # Result:
database executes authorization as a native query filter # Zero post-retrieval scanning needed –
authorization happens in DB layer

# Python: OPA partial evaluation → SQL filter import requests import json def
get_sql_filter_for_user(canonical_claims: dict) -> str: # Call OPA partial evaluation endpoint
response = requests.post( 'http://opa-sidecar:8181/v1/compile', json={ 'query':
'data.documents.allow == true', 'input': { 'user': { 'capabilities':
canonical_claims['capabilities'], 'clearance_level':
canonical_claims['principal'].get('clearance_level', 1), 'tenant_id':
canonical_claims['organization']['tenant_id'], } }, 'unknowns': ['input.document'] # Partial
eval on document attributes } ) partial_result = response.json() # Translate OPA residual to SQL
conditions = extract_conditions(partial_result) return f"WHERE {' AND '.join(conditions)}" #
OpenSearch equivalent – compile to DSL filter: def
get_opensearch_filter_for_user(canonical_claims: dict) -> dict: clearance =
canonical_claims['principal'].get('clearance_level', 1) tenant =
canonical_claims['organization']['tenant_id'] return { "bool": { "must": [ {"range":
{"classification": {"lte": clearance}}}, {"term": {"tenant_id": tenant}}, {"term":
{"embargo_active": False}}, ] } }
```

4.3 OPA WASM for Edge Authorization

Rego policies can be compiled to WebAssembly (WASM), enabling policy enforcement at the CDN/edge layer — before requests reach the origin — with sub-millisecond evaluation and zero network I/O for the authorization

check:

```
# Compile Rego to WASM (CI/CD pipeline step) # opa build -t wasm -e data/authz/allow
policies/edge_authz.rego # Output: bundle.tar.gz containing policy.wasm # Edge function
(CloudFront/Lambda@Edge): import wasmtime # or @open-policy-agent/opa-wasm in Node.js def
edge_authorizer(request): # Load WASM module (cached after first load) engine =
get_opa_wasm_engine() # cached # Evaluate policy input_data = { "method": request.method,
"path": request.path, "jwt_claims": extract_jwt_claims(request.headers.get('Authorization')) }
result = engine.evaluate(input_data) if not result.get('allow', False): return { 'status':
'403', 'statusDescription': 'Forbidden', 'body': 'Authorization denied' } return request #
Forward to origin # WASM evaluation characteristics: # • Latency: <0.5ms (no network I/O) # •
Policy bundle: updated via CDN cache invalidation # • Works at CloudFront Edge (Lambda@Edge or
CloudFront Functions) # • Use cases: rate limiting, geo-blocking, basic capability check # •
Limitation: no external data lookup (self-contained policy only)
```

5. Policy Simulation, Shadow Evaluation & Testing

5.1 Shadow Evaluation Architecture

Shadow evaluation runs the new policy alongside the existing authorization system without changing production behavior. Mismatches are logged for investigation.

[illegible]

5.2 Policy Test Vector Framework

```
# Comprehensive test vector generator # Generates test cases covering: happy paths, denial
paths, # boundary conditions, temporal conditions, multi-tenant, edge cases
import boto3 from itertools import product
def generate_test_vectors():
    vectors = [] # Test matrix:
    capability × mfa × business_hours × risk_score
    capabilities = [ "can_approve_payment",
["can_view_financial_data"], [], # No capabilities ]
    mfa_states = [True, False]
    business_hours = [True, False]
    risk_scores = [0, 30, 70, 100]
    for caps, mfa, biz_hrs, risk in product(
capabilities, mfa_states, business_hours, risk_scores ):
        expected = compute_expected_decision(
caps, mfa, biz_hrs, risk, action='InvokeTool', tool='PaymentApprovalTool' )
        vectors.append({
'name': f"caps={caps},mfa={mfa},biz={biz_hrs},risk={risk}",
'principal': { 'type': 'BankAI::User', 'id': 'test-user',
'attributes': { 'capabilities': caps, 'mfaVerified': mfa,
'businessUnit': 'FINANCE', 'geography': 'GB' } },
'action': 'BankAI::Action::"InvokeTool"',
'resource': 'BankAI::Tool::"PaymentApprovalTool"',
'context': { 'businessHours': biz_hrs, 'riskScore': risk,
'tenantId': 'bank-prod' },
'expected': expected })
    return vectors
def run_test_suite(vectors: list, policy_store_id: str):
    avp = boto3.client('verifiedpermissions')
    results = {'passed': 0, 'failed': 0, 'failures': []}
    for vec in vectors:
        response = avp.is_authorized(
policyStoreId=policy_store_id,
principal=vec['principal'],
action={'actionType': 'BankAI::Action', 'actionId': 'InvokeTool'},
resource={'entityType': 'BankAI::Tool', 'entityId': 'PaymentApprovalTool'},
context={'contextMap': build_context_map(vec['context'])})
        actual = response['decision']
        if actual == vec['expected']:
            results['passed'] += 1
        else:
            results['failed'] += 1
        results['failures'].append({ 'test': vec['name'],
'expected': vec['expected'], 'actual': actual,
'determining_policies': response.get('determiningPolicies', []) })
    coverage = results['passed'] / (results['passed'] + results['failed'])
    print(f"Coverage: {coverage:.1%} ({results['passed']}/{len(vectors)})")
    return results
```


6. Policy Governance Framework

6.1 Policy Review Board Structure

Role	Responsibility	Approval Required For
Policy Author (Developer)	Writes Cedar/Rego policies for new features or bug fixes	None — can open PR
Security Engineer	Reviews policies for security correctness, over-permissiveness, missing denies	Must approve all policy PRs
Data Privacy Officer	Reviews policies touching PII, classification, cross-border data	Must approve: PII-related, data export, classification change policies
IAM/Identity Team	Reviews claims normalization, role-capability mappings	Must approve: capability map changes, new role additions
Compliance Officer	Reviews policies for regulatory alignment (PCI, DORA, NIST)	Must approve: audit log policies, payment authorization policies
CISO (escalation)	Approves emergency policy changes, break-glass procedures	Required: production emergency hotfix, policy rollback of >100 policies

6.2 Policy Drift Detection

```
# AWS Config Rule: detect unauthorized AVP policy changes # config-rule.py — Lambda for custom
Config rule
import boto3
import json
avp = boto3.client('verifiedpermissions')
config = boto3.client('config')
cloudwatch = boto3.client('cloudwatch')
POLICY_STORE_ID = 'ps-XXXXXXXXXXXXXXXXX'
APPROVED_POLICY_HASHES_TABLE = 'approved-policy-hashes'

def handler(event, context):
    # Triggered by CloudTrail PutPolicy event via Config invocation
    invocation = json.loads(event['invokingEvent'])
    if invocation.get('configurationItemStatus') == 'OK':
        check_policy_drift()

def check_policy_drift():
    # Get all current policies in AVP
    policies = list_all_policies()
    # Compare against approved state in DynamoDB
    approved = get_approved_policy_state()
    violations = []
    for policy in policies:
        policy_id = policy['identifier']['policyId']
        policy_hash = compute_hash(policy['definition'])
        if policy_id not in approved:
            violations.append({'type': 'UNAUTHORIZED_NEW_POLICY', 'policyId': policy_id})
        elif approved[policy_id] != policy_hash:
            violations.append({'type': 'UNAUTHORIZED_MODIFICATION', 'policyId': policy_id, 'expectedHash': approved[policy_id], 'actualHash': policy_hash})
    # Check for deleted approved policies
    for policy_id in approved:
        if policy_id not in {p['identifier']['policyId'] for p in policies}:
            violations.append({'type': 'UNEXPECTED_DELETION', 'policyId': policy_id})
    if violations:
        # Send CloudWatch alarm
        cloudwatch.put_metric_data(
            Namespace='EnterpriseAuth/PolicyDrift',
            MetricData=[{
                'MetricName': 'PolicyViolations',
                'Value': len(violations),
                'Unit': 'Count'
            }]
        )
        # Send to Security Hub as finding
        publish_security_finding(violations)
    # Page on-call trigger
    pagerduty_alert(violations)
```